

Accurate Bounded Model Checking for OSEK/VDX Applications in CBMC

CBMC ISR/OSEK Research Branch

June 20, 2026

Abstract

OSEK/VDX applications consist of multiple tasks, a deterministic scheduler, a ready queue, and standardized application interfaces (APIs). If existing model checking methods for concurrent software are applied directly to such applications, a large number of interleavings of tasks may be checked although the running task is explicitly determined by the scheduler. The result is an approximate verification in which unnecessary interleavings of tasks may lead to a spurious bug. This report describes a CBMC-based bounded model checking flow that takes the behaviours of OSEK/VDX scheduler into account and reduces unnecessary interleavings before source-to-source instrumentation.

The first part uses OIL configuration and scheduler-related APIs so that task states, the ready queue, and task execution order follow the OSEK/VDX model during verification. The second part computes a manifest of candidate program points whose accesses may conflict with globals written by the interruption code, and then inserts guarded nondeterministic calls only at those points. On the public benchmark subset in this repository, the checked-in evidence contains 43 measured cases: 17 same-outcome comparable cases, 10 cases audited as `improved_correct`, 6 `no_injection_false` cases, 3 `needs_manual_review` cases, and 7 `pending_manual_review` cases. Representative public cases show that the method can remove unnecessary interleavings that produce spurious bugs and can also expose error traces that are not reached by the stock baseline. The added accuracy costs extra analysis and rebuild time, but it yields a more accurate bounded model checking workflow for OSEK/VDX applications.

1 Introduction

OSEK/VDX is an operating-system standard for vehicle-mounted software [1]. An application developed on OSEK/VDX OS consists of multiple tasks. To concurrently execute multiple tasks on one CPU, OSEK/VDX scheduler uses task states, priorities, and a ready queue to determine the running task. In contrast with many forms of concurrent software in which the running thread is not explicitly controlled by the application, OSEK/VDX applications are dispatched by a deterministic scheduler. OSEK/VDX OS also provides standardized application interfaces (APIs), such as `ActivateTask`, `TerminateTask`, `SetEvent`, and `WaitEvent`, through which tasks can dynamically change task states, events, and the task execution order.

To ensure the reliability of OSEK/VDX applications, the behaviours of the OSEK/VDX scheduler should be taken into account in the verification process. Testing is an important checking technique used in industry, but it is difficult to exhaustively check OSEK/VDX applications using testing technique, especially because of incomplete test-cases. In addition, it is hard to reproduce an error trace when an error is detected by testing, due to the concurrency of OSEK/VDX applications.

The main difficulty is accuracy. Existing model checking methods for concurrent software usually check all of the interleavings of threads. If such methods are applied directly to OSEK/VDX applications, the verification process becomes too imprecise because the behaviours

of OSEK/VDX scheduler are not taken into account. As a result, a large number of unnecessary interleavings of tasks may be checked in the verification stage, which can lead to spurious bugs and increases the checking costs in terms of time and memory. Therefore, a more accurate bounded model checking approach for OSEK/VDX applications should take into account the deterministic scheduler, the ready queue, and the standardized application interfaces (APIs).

This report describes a CBMC-based checking approach that addresses this problem in two parts. First, the APIs invoked from tasks are taken into account during the verification process, so that the states of tasks, synchronization events, shared resources, and task execution order can be handled according to the behaviours of the OSEK/VDX scheduler. Second, the checking model is constructed to avoid exploring unnecessary behaviours in the verification stage.

The report makes three contributions:

- It describes how bounded model checking can be specialized for OSEK/VDX applications whose tasks are dispatched by a deterministic scheduler.
- It introduces a manifest for building a new checking model with interrupt-service-routine calls only at manifest-listed program locations.
- It presents benchmark results showing that the proposed approach reduces unnecessary task interleavings and removes spurious bugs reported by the baseline.

The rest of this report is structured as follows. Section 2 presents the OSEK/VDX execution model and the role of bounded model checking. Section 3 describes the proposed checking approach. Section 4 reports the benchmark results and discusses the accuracy and checking costs of the approach. Section 5 summarizes the main limitations of the current implementation.

2 Background

2.1 OSEK/VDX Task, Scheduler, and Event

An application developed on OSEK/VDX OS consists of multiple tasks [1]. OSEK/VDX OS can process two types of tasks: basic tasks and extended tasks. The states of a basic task are running, ready, and suspended. Compared with a basic task, an extended task can hold synchronization events and has an additional waiting state. OSEK/VDX OS also provides standardized application interfaces (APIs), and tasks within the application can invoke these APIs to dynamically change the states of tasks, synchronization events, and shared resources.

OSEK/VDX executes multiple tasks on one CPU using a deterministic scheduler based on static priorities and a ready queue.

Synchronization events are used to implement synchronous behaviours between tasks. Only extended tasks can hold events, and these events are used as criteria for switching task states from running state to waiting state or from waiting state to ready state. The event process module responds to three APIs: `SetEvent`, `WaitEvent`, and `ClearEvent`. For example, when a running task invokes `WaitEvent` to wait for an event, it cannot continue until that event is set by another basic or extended task through `SetEvent`.

2.2 Model Checking and Bounded Model Checking

Model checking is an efficient and exhaustive verification technique for checking whether a target system satisfies a given property. The key idea is to construct a model that describes the executions of the target system and then search the constructed model with the given property. If the property is violated, model checking reports a trace that shows how the error can occur. This trace is usually named a counterexample, and it helps developers reproduce the faulty behaviour.

Bounded model checking follows the same verification goal, but it restricts the checking process with a specified bound. Instead of generating the full state space at once, bounded model checking constructs a transition system within the bounded verification space and uses a solver to check whether the property is violated. If a violation is found, the solver can report a counterexample. This makes bounded model checking useful for systems with large state spaces while still preserving the practical value of an error trace.

2.3 Checking Approaches for OSEK/VDX Applications

The execution characteristics of OSEK/VDX applications make their verification different from general concurrent software verification. In OSEK/VDX applications, the running task is explicitly determined by the OSEK/VDX scheduler according to the ready queue and configuration data. At the same time, APIs invoked from tasks may change the scheduling of tasks, for example by causing a context switch or by leading to different task execution orders at different branches. Therefore, the key problem is how to construct an accurate checking model that represents the executions of the application and handles the behaviours of OSEK/VDX OS.

Existing checking approaches for OSEK/VDX applications can be summarized as follows:

- **Spin-based checking approach.** This approach constructs a combination model of the application model and the OSEK/VDX OS model. In this model, tasks within the application are represented as processes, while the OSEK/VDX OS model dispatches tasks and responds to the APIs invoked from tasks. This approach can accurately simulate the executions of OSEK/VDX applications in the real OSEK/VDX OS. However, its scalability is limited because the behaviours of the OSEK/VDX OS model are also explored in the verification stage, which may lead to state space explosion when the application invokes many APIs.
- **SMT-based bounded model checking approach.** This approach uses an execution path generator (EPG) to construct a transition system for the OSEK/VDX application. The OSEK/VDX OS model is embedded in the generation process to dispatch tasks and respond to the APIs invoked from tasks. Therefore, the OS model does not need to be explored directly in the verification stage. This approach is more efficient and scalable than the Spin-based checking approach for applications with many tasks and APIs. However, it may become inefficient when the application contains many branches, because many execution paths have to be explored to construct the transition system.
- **Sequentialization-based checking approach.** This approach translates an OSEK/VDX application into a sequential model and then applies existing model checkers such as Spin or CBMC to verify the generated model. During the sequentialization process, the scheduling behaviours of OSEK/VDX OS are explicitly performed, and the APIs invoked from tasks are handled during translation. This approach enables existing model checkers to verify OSEK/VDX applications by checking a sequential model instead of a concurrent model.

Among the checking approaches summarized above, this report follows the SMT-based bounded model checking direction. In this direction, the key work is to construct a transition system that correctly reflects the executions of the target application. For OSEK/VDX applications, this transition system must take into account the deterministic scheduler, the ready queue, and the APIs invoked from tasks. This report uses CBMC as the bounded model checker and refines the checking process so that OSEK/VDX-specific scheduling behaviours are represented before verification.

2.4 Baseline CBMC Architecture

CBMC is a bounded model checker for C programs [2]. Its checking process can be viewed in three main stages. First, the front-end and `goto-cc` translate C source files into a GOTO

program. Second, `goto-symex` symbolically executes the GOTO program and constructs an SSA-style equation. Third, the solver backend checks the generated equation and, when a property is violated, CBMC can reconstruct a counterexample trace.

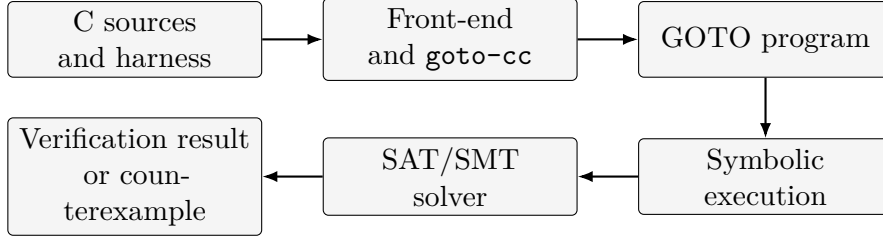


Figure 1: Baseline CBMC flow before the OSEK-aware and manifest-driven extensions described in this report.

This architecture is a strong baseline for C verification because bounded model checking can construct a transition system under a given bound and use a solver to check safety properties. However, this baseline is not sufficient by itself for OSEK/VDX applications. The limitation is not that CBMC lacks symbolic execution. The limitation is that the baseline CBMC flow does not automatically model the behaviours of the OSEK/VDX scheduler. In particular, APIs invoked from tasks can change task states, synchronization events, shared resources, and the ready queue. If these behaviours are not represented in the checking model, the verification process may not reflect the task execution order determined by the OSEK/VDX scheduler.

3 Methodology

3.1 Overview of the Proposed Approach

The proposed approach builds on the baseline CBMC architecture described in Section 2 and summarized in Figure 1. It does not replace bounded model checking, symbolic execution, or the solver backend. Instead, it changes how the checking model is constructed before final verification. The goal is to make the model more accurate for OSEK/VDX applications, where task execution order is determined by the deterministic scheduler, the ready queue, task states, and the APIs invoked from tasks.

The approach has two main parts. The first part is handling OSEK/VDX scheduler behaviours. During verification, the checking model must represent how OSEK/VDX APIs change task states, synchronization events, shared resources, and the ready queue. This allows the next running task to follow the behaviours of the OSEK/VDX scheduler rather than ordinary C control flow alone.

In terms of the first part, during verification, the checking model must represent how OSEK/VDX APIs change task states, synchronization events, shared resources, and the ready queue. This allows the next running task to follow the behaviours of the OSEK/VDX scheduler rather than ordinary C control flow alone.

The second part is manifest-guided interrupt service routine (ISR) insertion. In this report, an ISR is a function that may interrupt the normal execution of tasks. A naive model may insert ISR calls at many program locations by using `__CPROVER_ASYNC_X`, which can create unnecessary interleavings and increase the checking cost. To avoid this, the proposed approach first computes a manifest, which records the selected program locations where ISR calls should be inserted. The source tree is then rewritten according to this manifest, producing a new checking model for CBMC.

Table 1 summarizes the difference between the baseline CBMC flow and the proposed approach.

Aspect	Baseline CBMC	Proposed Approach
Task execution order	Follows the ordinary C program model unless OSEK/VDX scheduling is encoded separately.	Follows task states, the ready queue, and the deterministic scheduler inside the checking model.
OSEK/VDX APIs	May be treated as ordinary function calls if no OSEK/VDX-specific model is provided.	Are handled as operations that can change task states, synchronization events, shared resources, and the ready queue.
Interrupt behaviour	A broad model may allow ISR calls at many program locations.	ISR calls are inserted only at selected locations recorded in the manifest.
Interleavings	May include unnecessary interleavings that increase the checking cost and may lead to spurious bugs.	Reduces unnecessary interleavings before the final bounded model checking step.
Checking model	Represents the program after the standard CBMC front-end and symbolic execution flow.	Represents the OSEK/VDX scheduler, changes caused by APIs invoked from tasks, and selected ISR insertion points before final verification.

Table 1: Conceptual comparison between baseline CBMC and the proposed approach.

The key point is that the proposed approach keeps CBMC as the bounded model checker, but changes the model that CBMC receives. Instead of verifying a model that treats scheduling and interrupt effects too broadly, the approach constructs a checking model that better reflects the executions of OSEK/VDX applications.

The remainder of this section explains the approach in three steps. ?? describes how OSEK/VDX scheduler behaviours and APIs invoked from tasks are represented. ?? explains how the manifest is used to reduce unnecessary ISR insertion points. ?? explains how the checking model that takes the OSEK/VDX scheduler into account and the manifest-guided insertion step are combined before CBMC performs final verification.

To keep the discussion concrete, the remainder of this section uses one synthetic running example. *Task1* is a higher-priority extended task that waits for an event. *Task2* is a lower-priority task that can run while *Task1* is in waiting state. *ISR1* updates shared state and sets the event awaited by *Task1*. The example is intentionally small, but it is large enough to show the three main decisions in the methodology: how task execution order is determined, how unnecessary interleavings are reduced, and how the final checking model is built.

3.2 Taking the OSEK/VDX Scheduler into Account

The first methodological step is to represent the behaviour of the OSEK/VDX scheduler inside the checking model. In an OSEK/VDX application, the next running task is not chosen by an unstructured interleaving of threads. It is chosen by a deterministic scheduler that depends on task states, priorities, events, and the ready queue. If the checking model ignores those elements, the resulting task execution order no longer matches the execution order of the application.

Listing 1 shows the running example used throughout this section. The example combines waiting, event signaling, shared-state update, and an local computation so that each later step can be explained with reference to the same program.

Listing 1: Pseudo Code

```

int shared_ready = 0;
int shared_value = 0;

TASK(Task1)
{

```

```

    EventMaskType observed = 0;
    WaitEvent(EVENT_SHARED_READY);
    GetEvent(TASK_1_ID, &observed);
    if ((observed & EVENT_SHARED_READY) != 0u && shared_ready) {
        int sample = shared_value;
        assert(sample >= 0);
    }
    ClearEvent(EVENT_SHARED_READY);
    TerminateTask();
}

TASK(Task2)
{
    int checksum = 0;
    checksum = checksum + 1;
    checksum = checksum * 2;
    TerminateTask();
}

ISR(ISR1)
{
    shared_value = 7;
    shared_ready = 1;
    SetEvent(TASK_1_ID, EVENT_SHARED_READY);
}

```

Table 2 summarizes the intended execution context for this example. *Task1* starts as the running task, *Task2* is already ready with a lower priority, and *ISR1* can make *Task1* ready again by setting the event that *Task1* is waiting for.

Entity	Priority / type	Initial status and role
<i>Task1</i>	Higher-priority extended task	Running at the beginning of the path; waits for <code>EVENT_SHARED_READY</code> ; later reads the event state and the shared value.
<i>Task2</i>	Lower-priority task	Ready while <i>Task1</i> is running; may execute while <i>Task1</i> is in waiting state; performs only local computation in this example.
<i>ISR1</i>	Interrupt service routine	Writes <code>shared_ready</code> and <code>shared_value</code> , then sets the event awaited by <i>Task1</i> .

Table 2: Configuration summary for the running example.

The central point is that the standardized APIs in Listing 1 cannot be treated as ordinary function effects. When *Task1* executes `WaitEvent`, the important effect is not only that a C function is called. The important effect is that *Task1* may move from running state to waiting state. Once this happens, the scheduler may select *Task2* as the next running task because *Task2* is already in ready state.

Later, when *ISR1* executes `SetEvent`, the event state changes and *Task1* may move from waiting state back to ready state. Because *Task1* has higher priority than *Task2*, the deterministic scheduler may dispatch *Task1* before *Task2* continues. Therefore, the checking model must represent changes to task states, events, and the ready queue, not only ordinary control-flow changes inside the current C function.

Table 3 is the output of this first methodological step. It shows how task states, the ready queue, and the event state change when the same source program is checked under the OSEK/VDX model.

Step	Current context	Task states	Ready queue	Event state	Scheduler action
0	Start of path	<i>Task1</i> : running; <i>Task2</i> : ready	<i>Task2</i>	Clear	Continue in <i>Task1</i>
1	<i>Task1</i> executes WaitEvent while the event is clear	<i>Task1</i> : waiting; <i>Task2</i> : ready	<i>Task2</i>	Clear	Dispatch <i>Task2</i>
2	<i>Task2</i> executes local computation	<i>Task1</i> : waiting; <i>Task2</i> : running	Empty	Clear	Continue in <i>Task2</i> unless <i>ISR1</i> occurs
3	<i>ISR1</i> updates shared state and executes SetEvent	<i>Task1</i> : ready; <i>Task2</i> : running before dispatch	<i>Task1</i>	Set	Dispatch <i>Task1</i> because it has higher priority
4	<i>Task1</i> continues, observes the event, and reads the shared value	<i>Task1</i> : running; <i>Task2</i> : ready	<i>Task2</i>	Set, then cleared	Continue in <i>Task1</i>

Table 3: State evolution for the running example.

Figure ?? gives a high-level view of how scheduling information is produced in the checking model. When a task invokes an OSEK/VDX API, the API may change task states or events. These changes update the ready queue, and the deterministic scheduler uses the ready queue to decide which task should execute next. Therefore, the checking model must represent this cycle explicitly instead of treating API calls as ordinary function calls.

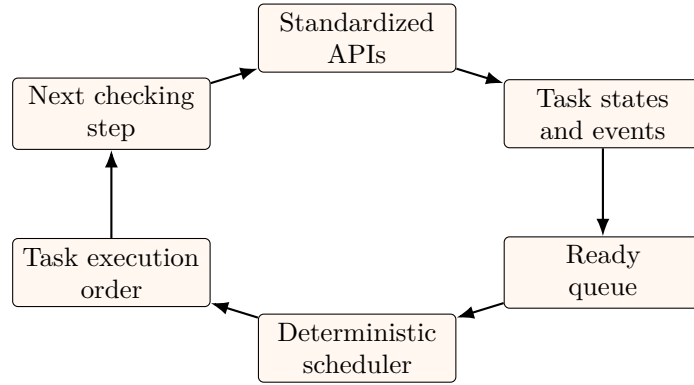


Figure 2: Conceptual cycle from standardized application interfaces (APIs) to task states, the ready queue, the deterministic scheduler, and the next checking step.

Read clockwise, some OSEK/VDX APIs change task states or event state. For example, **WaitEvent** may move an extended task from running state to waiting state when the awaited event is not set, while **SetEvent** may make a waiting task ready again by setting the awaited event. Other APIs, such as **GetEvent**, read event information that is already stored in the checking model. After each relevant API action, the checking model updates the task states, the event state, and the ready queue. The deterministic scheduler then uses this information to determine the next running task. That task execution order determines the next checking step, which may later reach another API call or another state update.

In the running example, this means that the verification path should not simply continue with the next statement in the C code after **WaitEvent** when the awaited event is clear. Instead, the checking model should represent that *Task1* moves to waiting state, that *Task2* may run while *Task1* is waiting, and that *ISR1* can set the event and make *Task1* ready again. In this way, the task execution order in the verification result can reflect the execution order determined by the OSEK/VDX scheduler.

3.3 Reducing Unnecessary Interleavings

After the checking model represents the OSEK/VDX scheduler, the next step is to control where interrupt service routine (ISR) calls are inserted. If ISR calls are inserted at too many program locations, the checking model may include many interleavings that do not affect the property being checked. These unnecessary interleavings can increase the checking costs in terms of states, time, and memory, and may also lead to spurious bugs.

Therefore, the proposed approach first identifies the program locations where ISR calls are relevant to the checked execution. These selected locations are recorded in a manifest. The later rewriting step uses this manifest to insert ISR calls only at those selected locations, instead of adding them broadly across the program.

The key question is therefore not “where could an ISR call be inserted in general?”, The key question is “where can an ISR call change the checking result?”. In the running example, *ISR1* is relevant when its updates to `shared_ready` and `shared_value`, or its call to `SetEvent`, can be observed later by *Task1* or by the scheduler.

If inserting *ISR1* at a program location cannot change a task state, event state, ready queue, or assertion result, then that insertion only adds an unnecessary interleaving to the checking model.

In the running example, *ISR1* has three visible effects: it writes `shared_value`, sets `shared_ready`, and calls `SetEvent(TASK_1_ID, EVENT_SHARED_READY)`. These effects matter when they can be used later by *Task1* or by the scheduler. For example, the call to `SetEvent` can move *Task1* from waiting state back to ready state, and the writes to `shared_ready` and `shared_value` can affect the later `if` statement and assertion in *Task1*.

By contrast, the checksum statements in *Task2* only update the local variable `checksum`. They do not read `shared_ready` or `shared_value`, they do not inspect the event state, and they do not invoke an OSEK/VDX API that changes task states or the ready queue. Therefore, these local statements do not create additional points.

Table 4 shows gives a readable summary of the selected program locations: where an *ISR1* call should be inserted, and where a separate insertion would only add an unnecessary interleaving.

Candidate program location	Can <i>ISR1</i> change what is read here?	Decision	Reason
Immediately before <code>WaitEvent(EVENT_SHARED_READY)</code>	Yes	Insert	<i>ISR1</i> can set <code>EVENT_SHARED_READY</code> . This can change whether <i>Task1</i> continues or moves to waiting state.
Immediately before <code>GetEvent(TASK_1_ID, &observed)</code>	Yes	Insert	The value stored in <code>observed</code> depends on whether <code>EVENT_SHARED_READY</code> has already been set.
Immediately before the <code>if</code> statement in <i>Task1</i>	Yes	Insert	The branch checks both <code>observed</code> and <code>shared_ready</code> , which can be changed by <i>ISR1</i> .
Immediately before the read of <code>shared_value</code>	Yes	Insert	The value read by <i>Task1</i> is written by <i>ISR1</i> .
Immediately before <code>checksum = checksum + 1</code> in <i>Task2</i>	No	Skip	This statement only updates the local variable <code>checksum</code> . It does not read the event state, <code>shared_ready</code> , or <code>shared_value</code> .
Immediately before <code>checksum = checksum * 2</code> in <i>Task2</i>	No	Skip	This statement is also local to <i>Task2</i> . A separate insertion here would add another interleaving without adding a new observation point in this example.

Table 4: Summary of selected ISR insertion locations in the running example.

The point of Table 4 is not to predict exactly when *ISR1* will occur. The point is to decide where inserting an *ISR1* call is useful for the checking model. The selected locations are the

places where *ISR1* can change what *Task1* or the scheduler observes, such as the event state, `shared_ready`, `shared_value`, or the next running task.

The skipped locations are different. In this example, the checksum statements in *Task2* only update the local variable `checksum`. They do not read the event state, they do not read `shared_ready` or `shared_value`, and they do not change task states or the ready queue. Skipping separate ISR insertions at those locations therefore reduces unnecessary interleavings without removing behaviour that can affect the checked path.

3.4 Building the Final Checking Model

The proposed approach combines the previous two steps to build the final checking model. First, it selects the program locations where an *ISR1* call can change what *Task1* or the scheduler observes. These locations are recorded in the manifest. Second, it uses the manifest to build a checking model in which ISR calls appear only at the selected locations.

This separation makes the method easier to control. The selection step answers the question: “where can an ISR call affect the event state, the ready queue, the shared variables, or the assertion checked by CBMC?” The model-building step answers a different question: how should the final checking model represent only those selected ISR calls?

As a result, the final checking model does not insert ISR calls at every possible program location. It keeps the locations that can affect the checked path and skips locations that only add unnecessary interleavings. This reduces the checking cost while preserving the behaviours needed for the running example.

For the running example, the selected locations can also be summarized in the compact form shown in Listing 2. This summary connects the selection step with the model-building step: it records where *ISR1* should be inserted, without showing low-level implementation details.

Listing 2: Readable summary of the selected ISR insertion locations for the running example.

Selected ISR insertion locations:
1. Before <code>WaitEvent(EVENT_SHARED_READY)</code>
2. Before <code>GetEvent(TASK_1_ID, &observed)</code>
3. Before the if statement that checks <code>observed</code> and <code>shared_ready</code>
4. Before the read of <code>shared_value</code>
Skipped locations: The local checksum updates inside <code>Task2</code>

Figure 3 shows how the final checking model is then built. The upper lane shows the baseline CBMC path. The lower lane shows the additional construction stages used by the proposed approach before final bounded model checking.

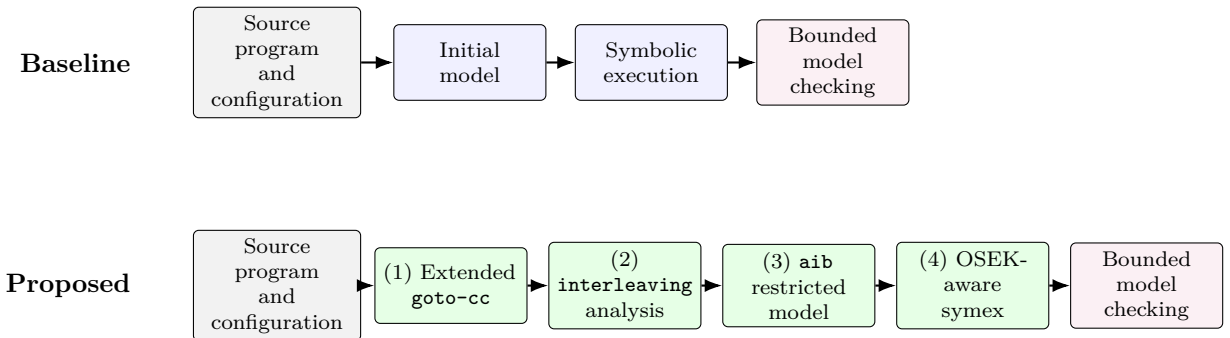


Figure 3: Baseline and improved checking-model construction. Numbered blocks indicate stages added by the proposed approach.

Conceptually, the upper lane starts from the original program and proceeds directly to model construction, symbolic execution, and bounded model checking. The lower lane inserts four additional responsibilities. The first added block builds a project-scoped view of the program rather than a single straight-line execution view. The second added block identifies where interruption-written state can affect the checked path. The third added block rebuilds a restricted source-level checking model that retains interruption-related behaviour only at those selected points. The fourth added block applies scheduler-consistent symbolic execution so that standardized APIs change task states, the ready queue, and task execution order in a way that matches OSEK/VDX semantics. This is the second major difference from baseline CBMC: the proposed approach changes both where interruption-related behaviour appears and how the resulting task execution order is interpreted during verification.

Listing 3 shows the output of the constructive step for the running example. The guarded calls are part of the checking model, not part of the original application program. They are placed only at the selected points. No such call is inserted inside the local checksum updates of *Task2*, because those updates were excluded as unnecessary interleavings in Table 4.

Listing 3: Illustrative restricted checking model for the running example.

```

TASK(Task1)
{
    EventMaskType observed = 0;
    if (nondet_bool()) ISR_Sensor(0);
    WaitEvent(EVENT_SENSOR_READY);
    if (nondet_bool()) ISR_Sensor(0);
    GetEvent(TASK_ID_Control, &observed);
    if (nondet_bool()) ISR_Sensor(0);
    if ((observed & EVENT_SENSOR_READY) != 0u && sensor_ready) {
        if (nondet_bool()) ISR_Sensor(0);
        int sample = sensor_value;
        assert(sample >= 0);
    }
    ClearEvent(EVENT_SENSOR_READY);
    TerminateTask();
}

TASK(Task2)
{
    int checksum = 0;
    checksum = checksum + 1;
    checksum = checksum * 2;
    TerminateTask();
}

```

The methodological role of Listing 3 is straightforward. The checking model still admits interruption-related behaviour, but only at points where that behaviour can change the execution being checked. This preserves the main advantage claimed in Sections 1 and 2: fewer unnecessary interleavings without ignoring the OSEK/VDX scheduler or the shared state written by interruptions.

3.5 Implementation Note

The current branch realizes this methodology with a small set of focused extensions rather than a monolithic verifier rewrite. Table 5 summarizes the role of each concrete component and the artifact or checking-model effect it produces.

Figure 3 uses these component boundaries implicitly. The added blocks in the lower lane correspond, in order, to the project-scoped model-construction extension around `goto-cc`, the separate

Component	What it consumes and does	What it produces
<code>goto-cc</code> extension	Consumes the source program and configuration data, then builds the initial checking model while preserving the project-level view needed by the later analysis stage.	An initial checking model together with the project-scoped information needed to analyze interruption relevance.
<code>interleaving-analysis</code>	Consumes the initial checking model and the designated interruption sources, then identifies program points where interruption-written state can affect the checked path.	A selection artifact describing the retained program points for restricted reconstruction.
<code>aib</code> with <code>lib-interleaving-block</code>	Consumes the original project tree and the selection artifact, then rebuilds a restricted source-level checking model with guarded interruption calls only at the retained points.	A rewritten project view that is narrower than blanket interruption insertion.
<code>goto-symex</code> with <code>os-api</code>	Consumes the restricted checking model and the OSEK/VDX configuration, then realizes the effects of standardized APIs on task states, events, the ready queue, and dispatch decisions during symbolic execution.	Scheduler- consistent symbolic states for final bounded model checking.

Table 5: Concrete realization of the methodology in the current branch.

`interleaving-analysis` stage, the `aib` rewriting step built on `lib-interleaving-block`, and the `goto-symex/os-api` realization of OSEK/VDX API effects during symbolic execution.

Exact command-line spellings, manifest-field names, and concrete artifact formats are kept out of the methodology body and collected in Section A. That appendix is the right place for interface-level details once the theoretical flow in this section is understood.

4 Evaluation

This section uses only the public benchmark families in the repository. Internal benchmark families are intentionally excluded. The goal is not to claim that the revised flow is always faster than the stock baseline. The checked-in evidence shows the opposite in many cases because the manifest, injection, and rebuild stages add cost. The evaluation instead answers two narrower questions:

1. Do the public benchmark artifacts contain cases where the revised flow is judged more faithful than the stock asynchronous baseline?
2. What cost is paid for that increase in accuracy?

4.1 Public Benchmark Scope

The checked-in public report `check-src/benchmark.md` covers the five suites summarized in Table 6. Together they contribute 43 measured cases.

4.2 Public Correctness Summary

The public subset has 43 measured cases. The correctness counts in Table 7 are recomputed for that public subset only.

Suite	Cases	Headline-ready	Notes
icbmc-large	12	yes	Larger i-CBMC-derived public cases.
icbmc-timeout-30m	4	no	Public diagnostic cases kept outside the automatic suite.
icbmc-timeout-30m-blink-highmem	2	no	Public high-memory blink cases.
icbmc-timeout-30m-logger-stable	4	no	Public logger-focused diagnostic cases.
intabs-large	21	yes	Larger public IntAbs-derived cases.

Table 6: Public benchmark suites used in this section.

Benchmark verdict	Cases	Interpretation
same_outcome_comparable	17	Stock and improved reached the same verification class.
improved_correct	10	The improved pipeline is audited as the more faithful model for the case.
no_injection_false	6	The improved path reported no injection candidates even though state written by an interleaving function can still affect the main-reachable state.
needs_manual_review	3	Output mismatch exists, but the current audit is not strong enough to choose a correct variant.
pending_manual_review	7	Mismatch exists and no verdict rule has yet classified the case.

Table 7: Correctness summary for the public benchmark subset only.

Two points stand out. First, the improved pipeline is not merely different from the stock baseline; it is often audited as more faithful. Second, the benchmark report explicitly keeps manual-review categories instead of silently forcing every mismatch into a win/loss label. That is important engineering hygiene for an evolving verification branch.

4.3 Representative Public Cases

Three public cases illustrate the different ways in which the revised flow changes the verification result.

icbmc-timeout-30m-logger-stable / logger2-bug-conc-cprover. The stock baseline reports *VERIFICATION FAILED* in 56 233 ms with 298.2 MB peak RSS, while the improved pipeline reports *VERIFICATION SUCCESSFUL* in 2017 ms with 16.6 MB peak RSS. The checked-in audit explains the difference directly: the stock async baseline exposes a non-atomic asynchronous interleaving in the logger update, whereas the improved pipeline checks the extracted interruption entry as one atomic region. This is a clear public example of unnecessary interleavings producing a spurious bug.

intabs-large / i8xx-tco-1. The stock baseline reports *VERIFICATION FAILED* in 1202 ms with 13.1 MB peak RSS, while the improved pipeline reports *VERIFICATION SUCCESSFUL* in 3395 ms with 15.3 MB peak RSS. The benchmark audit again points to unnecessary interleavings in the stock model. The stock run allows a non-atomic asynchronous interleaving, whereas the improved pipeline keeps the interruption entry atomic and narrows the candidate set to three guarded interleavings. This case shows the main accuracy gain of the revised flow even though the full measured time grows from 1202 ms to 3395 ms.

intabs-large / sc520wdt-2. The stock baseline reports *VERIFICATION SUCCESSFUL* in 1172 ms with 41.0 MB peak RSS. The improved pipeline reports *VERIFICATION FAILED* in 3559 ms with 16.7 MB peak RSS. Here the improved pipeline injects closer/writer interleavings

and reaches a violated property that the stock baseline proves away. This is the opposite public pattern from the previous two cases: the revised flow is slower, but it exposes an error trace that is not reached by the stock model. The full measured time rises from 1172 ms to 3559 ms, but the peak RSS drops from 41.0 MB to 16.7 MB.

Taken together, these public cases show the main cost of the revised flow. Even when the verification phase itself is not dramatically slower, the extra manifest, injection, and rebuild phases raise the full measured time. The public evidence therefore supports a clear trade-off: the revised flow aims at a more accurate verification result, not at a faster one.

For the use case of this report, that trade is acceptable. The objective is not to outperform the stock baseline on total wall-clock time, but to reduce unnecessary interleavings and make bounded model checking more accurate for OSEK/VDX applications.

5 Discussion and Limitations

The revised flow is deliberately conservative in some places and intentionally incomplete in others. Those limitations are not incidental; they define where future work should focus.

5.1 What the Current Design Gets Right

The strongest engineering choice in the branch is the separation between the behaviours of OSEK/VDX scheduler and the placement of interruption-related instrumentation. The task execution order is determined during verification, while the candidate program points are determined by an explicit analysis step and recorded in a manifest. This keeps the method maintainable. A change to OSEK/VDX scheduling policy does not automatically require a change to the instrumentation step, and a change to the candidate-line heuristics does not automatically require a change to the verification core.

The evidence in Section 4 also suggests that the extra structure is not just architecturally neat; it matters to outcomes. The benchmark report contains multiple cases in which the improved pipeline is audited as the more faithful model even though it is slower.

5.2 Current Analysis Limits

- **Source-file-driven target discovery.** Interleaving functions are selected from an explicit source-file list. If the relevant interruption file is not part of the build, the analysis cannot discover it.
- **Global-write-centric conflict rule.** Candidate lines are derived from globals written by the interleaving function and read or written by the main verification path. This keeps the rule simple, but it ignores purely read-side interactions by the interleaving function.
- **.c-only candidate filtering under project scope.** When project-root filtering is enabled, candidate recording is restricted to .c files under the root. Generated or indirect artifacts outside that slice are ignored.
- **Hand-written manifest IO.** Both the manifest writer and the aib reader are hand-coded. There is no schema version field, which makes coordinated changes brittle.

5.3 Current OSEK Model Limits

- **Partial OIL subset.** The parser currently handles the fields needed by the implemented scheduling rules: TASK, PRIORITY, SCHEDULE, AUTOSTART, TYPE, and EVENT_MASK.
- **AUTOSTART is metadata, not boot behavior.** The configuration records AUTOSTART, but the current flow still expects explicit task activation from the harness or runtime stubs.

- **WaitEvent is an approximation.** The implementation models the block / no-block decision and wake-up dispatch, but not every possible continuation detail of the full operating-system behaviour.
- **API coverage is intentionally narrow.** The current branch focuses on task and event services relevant to the public benchmarks. Resource, alarm, and interruption-related APIs beyond this subset are not yet modeled through the same boundary.

5.4 Performance Trade-offs

The benchmark report consistently shows that the revised flow pays an overhead tax. Even public cases add a manifest phase, an injection phase, and a rebuild. Larger cases can still be worth that cost when the stock baseline either misses a real bug or invents one through unnecessary interleavings, but the branch is not yet an optimization pass.

This suggests a practical reading of the current status: the branch is already useful as a more accurate bounded model checking workflow, but not yet as a drop-in faster replacement for stock CBMC. Improving analysis precision without paying so much orchestration cost is a natural next step.

6 Conclusion

This report documented a CBMC-based bounded model checking flow for OSEK/VDX applications. The central problem is accuracy: if the deterministic scheduler is ignored, many unnecessary interleavings of tasks may be superfluously checked and may result in a spurious bug. The revised flow addresses that problem by taking the behaviours of OSEK/VDX scheduler into account and by restricting interruption-related instrumentation through an explicit manifest.

The checked-in public benchmark evidence supports that choice. The revised flow is often slower because it adds analysis, instrumentation, and rebuild phases, but it is audited as the more faithful model in meaningful public cases. In particular, it removes unnecessary interleavings that produce spurious bugs and reaches some error traces that are not reached by the stock baseline.

The remaining work is also clear. The supported OSEK/VDX subset can be widened, the manifest contract can be made more robust, and the candidate-line rule can be made less dependent on a simple global-write heuristic. Even in its current form, however, the branch demonstrates a practical path toward more accurate bounded model checking for OSEK/VDX applications.

A Appendix: Concrete Interfaces

A.1 OSEK/VDX Checking Options

The methodology in Section 3 refers to OSEK/VDX-specific model construction without listing its command-line spellings. The following schematic command shows the documented options for selecting the OSEK/VDX checking model and the corresponding OIL configuration file.

Listing 4: Schematic OSEK/VDX option usage.

```
cbmc main.c --os-api osek --osek-oil app.oil
```


A.2 Example goto-cc Invocation

The following command illustrates the public project-mode flow on the `i8xx-tco-1` benchmark tree. It records the project root, names the interruption source file explicitly, and emits an interleaving manifest.

Listing 5: Example project-mode interleaving analysis command.

```
goto-cc \  
  check-src/benchmark-sources/intabs/cases/i8xx-tco-1/improved-pipeline/main.c \  
  check-src/benchmark-sources/intabs/cases/i8xx-tco-1/improved-pipeline/isr_define/isr.  
  .c \  
  --interleaving-project-root \  
    check-src/benchmark-sources/intabs/cases/i8xx-tco-1/improved-pipeline \  
  --interleaving-source-files \  
    check-src/benchmark-sources/intabs/cases/i8xx-tco-1/improved-pipeline/isr_define/  
    isr.c \  
  --interleaving-output \  
    check-src/benchmark-sources/intabs/cases/i8xx-tco-1/interleaving_pipeline.json \  
  -o check-src/benchmark-sources/intabs/cases/i8xx-tco-1/i8xx-tco-1.out
```

A.3 Example aib Invocation

Listing 6: Example AIB rewrite command.

```
aib \  
  check-src/benchmark-sources/intabs/cases/i8xx-tco-1/improved-pipeline \  
  check-src/benchmark-sources/intabs/cases/i8xx-tco-1/interleaving_pipeline.json \  
  out/i8xx-tco-1_injected \  
  out/i8xx-tco-1_injected/interleaving_pipeline_effective.json
```

A.4 Illustrative Guarded Call

The restricted checking model is created by inserting guarded interruption calls only at the selected program points. The example below is illustrative rather than benchmark-specific.

Listing 7: Illustrative guarded interruption call.

```
if (nondet_bool()) isr1(0);
```

A.5 Documented Selection-Artifact Fields

Field	Meaning
<code>project.project_root</code>	Root directory used for path scoping and report-path relativization.
<code>project.interleaving_source_files</code>	Source files whose functions are treated as interleaving functions.
<code>project.translation_units</code>	Effective build inputs associated with the initial checking model that produced the selection artifact.
<code>interleaving[*].name</code>	Interleaving function identifier recorded for rewriting.
<code>interleaving[*].write_global_var</code>	Global symbols that the interleaving function may write.
<code>interleaving[*].line_added_block_with_file</code>	Project-relative file and line pairs where guarded calls may be inserted.

Table 8: Documented fields of the selection artifact used between analysis and rewriting.

A.6 Manifest Excerpt

The manifest below is a schematic excerpt showing only the documented public fields. It illustrates how project metadata and interleaving candidates are separated.

Listing 8: Schematic manifest excerpt.

```
{
  "project": {
    "project_root": "/path/to/check-src/benchmark-sources/intabs/cases/i8xx-tco-1/
    improved-pipeline",
    "interleaving_source_files": ["isr_define/isr.c"],
    "translation_units": [
      "main.c",
      "isr_define/isr.c"
    ]
  },
  "interleaving": [
    {
      "name": "isr1",
      "write_global_var": ["<global_1>", "<global_2>"],
      "line_added_block_with_file": [
        { "file": "main.c", "line": [<line_a>, <line_b>] }
      ]
    }
  ]
}
```

This schema is the compatibility boundary between `interleaving-analysis` and `aib`; any field change must be implemented on both sides.

References

- [1] *OSEK/VDX Operating System Specification 2.2.1*. Document OS221.doc, January 16, 2003. OSEK Group. 2003.
- [2] Diffblue Ltd. *CBMC Architecture*. 2026. URL: <https://diffblue.github.io/cbmc/cbmc-architecture.html> (visited on 06/19/2026).